

Circular Queue

Array Representation, Operations and C Implementation

Dr. Siddhant Thapliyal

Department of Computer Science and Engineering
Graphic Era (Deemed to be University), Dehradun

Types of Queues

A queue can be classified into:

- ① Circular Queue
- ② Double-Ended Queue or Deque
- ③ Priority Queue
- ④ Multiple Queue

This lecture focuses on:

Circular Queue

Problem with a Linear Queue

Consider a linear queue of capacity 10.

$$front = 0, \quad rear = 9$$

54	9	7	18	14	36	45	21	99	72
0	1	2	3	4	5	6	7	8	9

Since:

$$rear = MAX - 1$$

no further insertion is possible.

False Overflow

Suppose two elements are deleted.

$$front = 2, \quad rear = 9$$

Deleted	Deleted	7	18	14	36	45	21	99	72
0	1	2	3	4	5	6	7	8	9

Positions 0 and 1 are free, but:

$$rear = MAX - 1$$

still prevents insertion.

Problem

Available memory exists, but the linear queue cannot reuse it.

Possible Solutions

Two possible solutions are:

- ① Shift all active elements towards index 0.
- ② Treat the array as circular.

Shifting requires:

$$O(n)$$

time.

A circular queue reuses empty positions in:

$$O(1)$$

time.

Possible Solutions

Two possible solutions are:

- ① Shift all active elements towards index 0.
- ② Treat the array as circular.

Shifting requires:

$$O(n)$$

time.

A circular queue reuses empty positions in:

$$O(1)$$

time.

Circular Queue is the better solution

What is a Circular Queue?

Definition

A circular queue is a queue in which the last array position is logically connected to the first array position.

After index:

$$MAX - 1$$

the next index becomes:

$$0$$

This circular movement is obtained using:

$$(index + 1) \bmod MAX$$

Circular Index Movement

Assume:

$$MAX = 5$$

The sequence of indices is:

$$0 \rightarrow 1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 0$$

Using modulo:

$$(0 + 1) \bmod 5 = 1$$

$$(3 + 1) \bmod 5 = 4$$

$$(4 + 1) \bmod 5 = 0$$

Circular Index Movement

Assume:

$$MAX = 5$$

The sequence of indices is:

$$0 \rightarrow 1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 0$$

Using modulo:

$$(0 + 1) \bmod 5 = 1$$

$$(3 + 1) \bmod 5 = 4$$

$$(4 + 1) \bmod 5 = 0$$

Modulo makes the index wrap to the beginning.

Array View of a Circular Queue

Assume:

$$MAX = 6, \quad front = 3, \quad rear = 1$$

50	60	Empty	20	30	40
0	1	2	3	4	5

Logical queue order is:

20, 30, 40, 50, 60

because traversal starts from 'front' and wraps after index 5.

Role of Front and Rear

- `front` identifies the first active element.
- `rear` identifies the last active element.
- Enqueue moves `rear` circularly.
- Dequeue moves `front` circularly.

The next rear position is:

$$(rear + 1) \bmod MAX$$

The next front position is:

$$(front + 1) \bmod MAX$$

Empty Condition

Initially:

$$front = -1$$

$$rear = -1$$

Therefore:

Queue is empty if $front = -1$

After deleting the only element:

$$front = rear$$

both are reset to:

$$-1$$

Full Condition

A circular queue is full when the next rear position is the front position.

$$(rear + 1) \bmod MAX = front$$

Examples for $MAX = 5$:

$$front = 0, rear = 4$$

because:

$$(4 + 1) \bmod 5 = 0$$

Another full arrangement is:

$$front = 3, rear = 2$$

because:

$$(2 + 1) \bmod 5 = 3$$

Why the Modulo Full Condition is Better

The condition:

$$front = 0 \quad \text{and} \quad rear = MAX - 1$$

detects only one full arrangement.

But a wrapped queue may also be full when:

$$front = 3, \quad rear = 2$$

Therefore, the general full condition is:

$$(rear + 1) \bmod MAX = front$$

Important

Use the modulo condition in a general circular-queue implementation.

Enqueue Operation

Enqueue inserts a value at the rear.

Three major cases are possible:

- 1 The queue is full.
- 2 The queue is empty.
- 3 The queue contains one or more elements.

For a non-empty queue:

$$rear = (rear + 1) \bmod MAX$$

Then:

$$queue[rear] = value$$

Enqueue Algorithm

Algorithm: ENQUEUE

① If:

$$(rear + 1) \bmod MAX = front$$

report overflow.

② If the queue is empty:

$$front = rear = 0$$

③ Otherwise:

$$rear = (rear + 1) \bmod MAX$$

④ Store:

$$queue[rear] = value$$

$O(1)$

First Enqueue

Initially:

$$front = rear = -1$$

Enqueue 25:

$$front = rear = 0$$

$$queue[0] = 25$$

The queue becomes:



$$front = 0, \quad rear = 0$$

Enqueue Without Wrapping

Assume:

$$MAX = 6, \quad front = 1, \quad rear = 3$$

Queue:

20, 30, 40

Enqueue 50:

$$rear = (3 + 1) \bmod 6 = 4$$

Store:

$$queue[4] = 50$$

Final queue:

20 30 40 50

Circular Queue

Enqueue with Wrapping

Assume:

$$MAX = 6, \quad front = 2, \quad rear = 5$$

The next rear position is:

$$rear = (5 + 1) \bmod 6$$

$$rear = 0$$

A new value is inserted at index 0.

Enqueue with Wrapping

Assume:

$$MAX = 6, \quad front = 2, \quad rear = 5$$

The next rear position is:

$$rear = (5 + 1) \bmod 6$$

$$rear = 0$$

A new value is inserted at index 0.

This reuses a position that became free after an earlier deletion.

Enqueue Function in C

```
1 int enqueue(int value)
2 {
3     if ((rear + 1) % MAX == front)
4     {
5         printf("Circular queue overflow\n");
6         return 0;
7     }
8
9     if (front == -1)
10    {
11        front = 0;
12        rear = 0;
13    }
14    else
15    {
16        rear = (rear + 1) % MAX;
17    }
18
19    queue[rear] = value;
20    return 1;
21 }
```

Deque Operation

Deque removes the value at the front.

Three cases are possible:

- 1 The queue is empty.
- 2 The queue contains only one element.
- 3 The queue contains multiple elements.

For multiple elements:

$$front = (front + 1) \bmod MAX$$

Deque Algorithm

Algorithm: DEQUEUE

① If:

$$front = -1$$

report underflow.

② Store:

$$value = queue[front]$$

③ If:

$$front = rear$$

reset:

$$front = rear = -1$$

④ Otherwise:

$$front = (front + 1) \bmod MAX$$

⑤ Return the removed value

Deleting the Only Element

Assume:

$$front = rear = 3$$

The only active element is:

$$queue[3]$$

After deletion:

$$front = rear = -1$$

The circular queue becomes empty.

Deque with Wrapping

Assume:

$$MAX = 6$$

$$front = 5, \quad rear = 2$$

Delete the value at index 5.

The new front is:

$$front = (5 + 1) \bmod 6$$

$$front = 0$$

The front wraps to the beginning of the array.

Deque Function in C

```
1 int dequeue(int *removedValue)
2 {
3     if (front == -1)
4     {
5         printf("Circular queue underflow\n");
6         return 0;
7     }
8
9     *removedValue = queue[front];
10
11    if (front == rear)
12    {
13        front = -1;
14        rear = -1;
15    }
16    else
17    {
18        front = (front + 1) % MAX;
19    }
20
21    return 1;
22 }
```

Peek Operation

Peek returns the front element without deletion.

If:

$$front = -1$$

the queue is empty.

Otherwise:

$$\text{Front value} = \text{queue}[front]$$

Peek does not change:

front

or:

rear

Time complexity:

Peek Function

```
1 int peek(int *frontValue)
2 {
3     if (front == -1)
4     {
5         printf("Circular queue is empty\n");
6         return 0;
7     }
8
9     *frontValue = queue[front];
10    return 1;
11 }
```

Displaying a Circular Queue

The queue may occupy two array segments:

$$front \dots MAX - 1$$

and:

$$0 \dots rear$$

A general traversal begins at 'front' and repeatedly performs:

$$i = (i + 1) \bmod MAX$$

Traversal ends after processing index 'rear'.

Display Function

```
1 void display(void)
2 {
3     int i;
4
5     if (front == -1)
6     {
7         printf("Circular queue is empty\n");
8         return;
9     }
10
11     printf("FRONT -> ");
12
13     i = front;
14
15     while (1)
16     {
17         printf("%d ", queue[i]);
18
19         if (i == rear)
20             break;
21
22         i = (i + 1) % MAX;
```

Size of a Circular Queue

If the queue is empty:

$$size = 0$$

If:

$$rear \geq front$$

then:

$$size = rear - front + 1$$

Otherwise:

$$size = MAX - front + rear + 1$$

A compact formula is:

$$size = ((rear - front + MAX) \bmod MAX) + 1$$

Size Function

```
1 int size(void)
2 {
3     if (front == -1)
4         return 0;
5
6     return ((rear - front + MAX) % MAX) + 1;
7 }
```

Examples:

$front = 1, rear = 3, MAX = 5$

$size = 3$

and:

$front = 3, rear = 1, MAX = 5$

Dry Run

Assume:

$$MAX = 5$$

Initially:

$$front = rear = -1$$

Perform:

ENQUEUE(10)

ENQUEUE(20)

ENQUEUE(30)

DEQUEUE()

Dry-Run Table

Step	Operation	Front	Rear	Logical Queue
0	Initial	-1	-1	Empty
1	Enqueue 10	0	0	10
2	Enqueue 20	0	1	10, 20
3	Enqueue 30	0	2	10, 20, 30
4	Dequeue	1	2	20, 30
5	Dequeue	2	2	30
6	Enqueue 40	2	3	30, 40
7	Enqueue 50	2	4	30, 40, 50
8	Enqueue 60	2	0	30, 40, 50, 60

Final Array after Wrapping

After the dry run:

$front = 2,$ $rear = 0$

60	Deleted	30	40	50
0	1	2	3	4

Logical queue order:

30, 40, 50, 60

Interactive Trace

Assume:

$$MAX = 4$$

Perform:

ENQUEUE(5)

ENQUEUE(10)

ENQUEUE(15)

DEQUEUE()

ENQUEUE(20)

Interactive Trace

Assume:

$$MAX = 4$$

Perform:

ENQUEUE(5)

ENQUEUE(10)

ENQUEUE(15)

DEQUEUE()

ENQUEUE(20)

Complexity Analysis

Operation	Time Complexity	Reason
Enqueue	$O(1)$	One circular index update
Dequeue	$O(1)$	One circular index update
Peek	$O(1)$	Direct front access
isEmpty	$O(1)$	One comparison
isFull	$O(1)$	One modulo operation
Size	$O(1)$	Arithmetic expression
Display	$O(n)$	Visits all active elements

Space complexity:

$$O(MAX)$$

Linear Queue vs Circular Queue

Feature	Linear Queue	Circular Queue
Memory reuse	Limited	Efficient
False overflow	Possible	Avoided
Rear movement	Only forward	Circular
Index update	$rear + 1$	$(rear + 1) \% MAX$
Implementation	Simpler	Slightly more complex
Enqueue	$O(1)$	$O(1)$
Dequeue	$O(1)$	$O(1)$

Definitions and Helper Functions

```
1 #include <stdio.h>
2
3 #define MAX 5
4
5 int queue[MAX];
6 int front = -1;
7 int rear = -1;
8
9 int isEmpty(void)
10 {
11     return front == -1;
12 }
13
14 int isFull(void)
15 {
16     return (rear + 1) % MAX == front;
17 }
18
19 int size(void)
20 {
21     if (isEmpty())
22         return 0;
```

Enqueue Implementation

```
1 int enqueue(int value)
2 {
3     if (isFull())
4     {
5         printf(
6             "Queue overflow: cannot insert %d\n",
7             value
8         );
9
10        return 0;
11    }
12
13    if (isEmpty())
14    {
15        front = 0;
16        rear = 0;
17    }
18    else
19    {
20        rear = (rear + 1) % MAX;
21    }
22 }
```

Deque Implementation

```
1 int dequeue(int *removedValue)
2 {
3     if (isEmpty())
4     {
5         printf("Queue underflow\n");
6         return 0;
7     }
8
9     *removedValue = queue[front];
10
11    if (front == rear)
12    {
13        front = -1;
14        rear = -1;
15    }
16    else
17    {
18        front = (front + 1) % MAX;
19    }
20
21    return 1;
22 }
```

Peek and Display

```
1 int peek(int *frontValue)
2 {
3     if (isEmpty())
4     {
5         printf("Queue is empty\n");
6         return 0;
7     }
8
9     *frontValue = queue[front];
10    return 1;
11 }
12
13 void display(void)
14 {
15     int i;
16
17     if (isEmpty())
18     {
19         printf("Queue is empty\n");
20         return;
21     }
22 }
```

Main Function: Menu

```
1 int main(void)
2 {
3     int choice;
4     int value;
5
6     do
7     {
8         printf("\n--- CIRCULAR QUEUE MENU ---\n");
9         printf("1. Enqueue\n");
10        printf("2. Dequeue\n");
11        printf("3. Peek\n");
12        printf("4. Display\n");
13        printf("5. Size\n");
14        printf("6. Check Empty\n");
15        printf("7. Check Full\n");
16        printf("8. Exit\n");
17        printf("Enter choice: ");
18
19        if (scanf("%d", &choice) != 1)
20        {
21            printf("Invalid input\n");
22            return 1;
```

Main Function: Operations

```
1  switch (choice)
2  {
3  case 1:
4      printf("Enter value: ");
5
6      if (scanf("%d", &value) != 1)
7      {
8          printf("Invalid value\n");
9          return 1;
10     }
11
12     if (enqueue(value))
13         printf("%d inserted\n", value);
14
15     break;
16
17 case 2:
18     if (dequeue(&value))
19         printf("Deleted value: %d\n", value);
20
21     break;
```

Main Function: Remaining Cases

```
1      case 4:
2          display();
3          break;
4
5      case 5:
6          printf("Queue size: %d\n", size());
7          break;
8
9      case 6:
10         printf(
11             isEmpty()
12             ? "Queue is empty\n"
13             : "Queue is not empty\n"
14         );
15         break;
16
17     case 7:
18         printf(
19             isFull()
20             ? "Queue is full\n"
21             : "Queue is not full\n"
22         );
```

Complete Program: Part 1

```
1 #include <stdio.h>
2
3 #define MAX 5
4
5 int queue[MAX];
6 int front = -1;
7 int rear = -1;
8
9 int isEmpty(void)
10 {
11     return front == -1;
12 }
13
14 int isFull(void)
15 {
16     return (rear + 1) % MAX == front;
17 }
18
19 int size(void)
20 {
21     if (isEmpty())
22         return 0;
```

Complete Program: Part 2

```
1 int dequeue(int *removedValue)
2 {
3     if (isEmpty())
4         return 0;
5
6     *removedValue = queue[front];
7
8     if (front == rear)
9         front = rear = -1;
10    else
11        front = (front + 1) % MAX;
12
13    return 1;
14 }
15
16 int peek(int *frontValue)
17 {
18     if (isEmpty())
19         return 0;
20
21     *frontValue = queue[front];
22     return 1;
```

Complete Program: Part 3

```
1 int main(void)
2 {
3     int choice;
4     int value;
5
6     do
7     {
8         printf("\n1. Enqueue");
9         printf("\n2. Dequeue");
10        printf("\n3. Peek");
11        printf("\n4. Display");
12        printf("\n5. Size");
13        printf("\n6. Exit");
14        printf("\nEnter choice: ");
15
16        if (scanf("%d", &choice) != 1)
17            return 1;
18
19        switch (choice)
20        {
21            case 1:
22                printf("Enter value: ");
```

Complete Program: Part 4

```
1      case 3:
2          if (peek(&value))
3              printf("Front: %d\n", value);
4          else
5              printf("Queue is empty\n");
6
7          break;
8
9      case 4:
10         display();
11         break;
12
13     case 5:
14         printf("Size: %d\n", size());
15         break;
16
17     case 6:
18         printf("Exiting\n");
19         break;
20
21     default:
22         printf("Invalid choice\n");
```

Applications of Circular Queues

Circular queues are used in:

- Round-robin CPU scheduling
- Circular buffers
- Audio and video streaming
- Keyboard buffers
- Network packet buffering
- Producer-consumer systems
- Traffic-light control
- Repeated resource allocation
- Memory-management systems

Round-Robin Scheduling

Suppose the ready queue contains:

P_1, P_2, P_3

After P_1 receives one time quantum, it is placed at the rear:

P_2, P_3, P_1

After P_2 executes:

P_3, P_1, P_2

The queue repeatedly cycles through the processes.

Circular Queue

Rapid-Fire Questions

- 1 What problem of a linear queue is solved by a circular queue?

Rapid-Fire Questions

- 1 What problem of a linear queue is solved by a circular queue?
- 2 Why is modulo used?

Rapid-Fire Questions

- ❶ What problem of a linear queue is solved by a circular queue?
- ❷ Why is modulo used?
- ❸ What is the empty condition?

Rapid-Fire Questions

- ❶ What problem of a linear queue is solved by a circular queue?
- ❷ Why is modulo used?
- ❸ What is the empty condition?
- ❹ What is the general full condition?

Rapid-Fire Questions

- 1 What problem of a linear queue is solved by a circular queue?
- 2 Why is modulo used?
- 3 What is the empty condition?
- 4 What is the general full condition?
- 5 When are front and rear reset to -1 ?

Rapid-Fire Questions

- ➊ What problem of a linear queue is solved by a circular queue?
- ➋ Why is modulo used?
- ➌ What is the empty condition?
- ➍ What is the general full condition?
- ➎ When are front and rear reset to -1 ?
- ➏ What is the time complexity of enqueue?

Multiple-Choice Question

For:

$$MAX = 6, \quad front = 2, \quad rear = 1$$

is the queue full?

- 1 Yes
- 2 No

Multiple-Choice Question

For:

$$MAX = 6, \quad front = 2, \quad rear = 1$$

is the queue full?

- 1 Yes
- 2 No

$$(rear + 1) \bmod MAX$$

$$= (1 + 1) \bmod 6$$

$$= 2 = front$$

Yes

Find the Error

```
1 void enqueue(int value)
2 {
3     if (rear == MAX - 1)
4         printf("Overflow");
5     else
6         queue[++rear] = value;
7 }
```

Why is this wrong for a circular queue?

Find the Error

```
1 void enqueue(int value)
2 {
3     if (rear == MAX - 1)
4         printf("Overflow");
5     else
6         queue[++rear] = value;
7 }
```

Why is this wrong for a circular queue?

It treats the queue as linear and does not reuse positions before 'front'.

Correct rear update:

$$rear = (rear + 1) \% MAX$$

Classroom Exercise

Assume:

$$MAX = 5$$

Perform:

ENQUEUE(10)

ENQUEUE(20)

ENQUEUE(30)

DEQUEUE()

DEQUEUE()

Classroom Exercise: Answer

Final values:

$front = 2$

$rear = 0$

Logical queue:

30, 40, 50, 60

The insertion of:

60

causes rear to wrap from index 4 to index 0.

Answer the following:

- 1 Explain false overflow.
- 2 Why is the condition $rear = MAX - 1$ insufficient?
- 3 Derive the circular queue full condition.
- 4 What happens after deleting the only element?
- 5 Find the next index after 7 when $MAX = 8$.
- 6 Compare linear and circular queues.

References

- ① Ellis Horowitz and Sartaj Sahni, *Fundamentals of Data Structures*, Computer Science Press.
- ② Ellis Horowitz, Sartaj Sahni and Susan Anderson-Freed, *Fundamentals of Data Structures in C*, Universities Press.
- ③ Reema Thareja, *Data Structures Using C*, Oxford University Press.

Questions?

Thank You